Differentiable Ray Tracing with Gaussians for Unified Radio Propagation Simulation and View Synthesis

Niklas Vaara[1], Lam Huynh[2], Pekka Sangi[1], Miguel Bordallo López[1], Janne Heikkilä[1]
[1] Center for Machine Vision and Signal Analysis, University of Oulu, Finland
[2] CubiCasa Oy, Finland
https://github.com/nvaara/GaussianRT-RF-NVS

Abstract

Explicit neural representations such as 3D Gaussian Splatting (3DGS) enable high-fidelity and real-time novel view synthesis, yet optimize for alpha-composited optical appearance rather than ray-intersectable geometry. In contrast, radio-frequency (RF) digital twins require deterministic multi-bounce paths, where the geometry dictates trajectories and their associated attenuation and delay. We introduce a framework enabling differentiable RF propagation simulation directly within visually reconstructed neural scenes, allowing point-to-point path computation between arbitrary 3D locations while preserving high-quality visual rendering. Unlike conventional RF simulation pipelines that rely on manually constructed meshes, we embed Gaussian primitives into a hardware-accelerated ray tracing structure as the underlying spatial representation. By extracting physically meaningful channel impulse responses from visual-only reconstructions, we provide cross-modal evidence that neural reconstructions can serve as unified spatial representations for both electromagnetic propagation simulation and photorealistic view synthesis.

1 Introduction

Cameras and radio transceivers provide complementary observations of the same physical scene: one through visible-light image formation, the other through radio wave propagation. Although these measurements appear very different, they are governed by the same underlying geometry and material interfaces: the surfaces that create image structure also block, reflect, and attenuate radio waves. This makes novel-view synthesis and radio-frequency (RF) simulation two forward models over a shared latent 3D environment.

Recent progress in neural scene representations has made this shared-view increasingly plausible. In particular, 3D Gaussian Splatting [1] has emerged as a highly efficient explicit representation for reconstructing scenes from images or videos, enabling high-fidelity, real-time novel-view synthesis without manual 3D modeling. Subsequent extensions [2–4] further improve geometric fidelity through depth supervision, normal regularization, and surface-aware constraints. However, despite their visual quality, standard 3DGS pipelines are designed around rasterization and alpha-composited optical appearance. They do not directly expose the ray-intersectable geometry required to compute physical multi-bounce propagation paths.

This limitation is especially consequential for wireless digital twins. Accurate radio propagation simulation requires deterministic line-of-sight (LoS) and non-line-of-sight (NLoS) multipath trajectories, since these paths determine the channel characteristics used to construct channel frequency responses (CFRs) and channel impulse responses (CIRs). Existing ray tracing (RT) pipelines therefore rely on explicit geometric models, typically CAD or mesh reconstructions, that are expensive to build, difficult to update, and often lack the fine geometric details present in real environments. As a result, the geometric modeling bottleneck remains a major obstacle to scalable, site-specific RF simulation.

Preprint.

The second obstacle is material uncertainty. Radio propagation modeling depends not only on geometry but also on material parameters such as permittivity and conductivity. In practice, these parameters are often incomplete, unavailable, or specified only for limited material classes and frequency ranges, as in standards such as ITU-R P.2040 [5]. This mismatch can lead to substantial discrepancies between simulated and measured channels. Recent differentiable RT methods [6, 7] address this issue by learning material properties from RF measurements, but their effectiveness still heavily depends on having an accurate geometric model of the environment.

These parallel advances reveal a critical problem: computer vision can now reconstruct rich 3D scenes automatically, while RF simulation can increasingly learn electromagnetic (EM) parameters from measurements, but the two remain disconnected by incompatible scene representations. Visual neural reconstructions provide appearance-optimized Gaussian primitives, whereas EM simulation requires deterministic multi-bounce paths over physically meaningful geometry. To bridge this divide, we introduce a differentiable RT framework built directly on Gaussian scene representations. Our method enables multi-bounce point-to-point path computation within visually reconstructed neural scenes, while preserving photorealistic novel-view synthesis and supporting differentiable EM computations that enables learning from RF measurements.

Our contributions are threefold: (1) a unified, differentiable Gaussian representation that jointly supports photorealistic rendering, multi-bounce geometric path tracing, and differentiable electromagnetic modeling; (2) a real-world multimodal dataset comprising RGB-D images, calibrated camera poses, and channel measurements; and (3) a differentiable alignment objective that optimizes channel parameters by matching simulated and measured CIRs. Together, these elements establish a single spatial representation that generalizes across optical and RF domains, enabling scalable, measurement-driven digital twins without manual geometric modeling.

2 Radio Propagation Preliminaries

In wireless channels, the received signal is typically composed of multiple propagation paths. These paths from transmitter (TX) to receiver (RX) are due to interactions with the environment via different mechanisms such as reflection. These paths fulfill Fermat's principle of least time, and can effectively be modeled with rays. Each path, in addition to the time delay τ_i, is modeled with a complex path coefficient a_i computed, similarly to [6], using

$$a_i = \frac{\lambda}{4\pi} \mathbf{G}_{rx}(\theta_{rx,i}, \phi_{rx,i})^{\mathsf{H}} \mathbf{T}_i \mathbf{G}_{tx}(\theta_{tx,i}, \phi_{tx,i}), \tag{1}$$

where λ is the wavelength and $\mathbf{G}_{tx}$ and $\mathbf{G}_{rx}$ are the antenna patterns of TX and RX, respectively. Both patterns are functions of the azimuth ϕ and elevation θ angles, with output in $\mathbb{C}^{2\times 1}$. The symbol $\mathbf{T}_i$ denotes the transfer matrix that contains the effect of all EM interactions of the ith path, including free-space path loss. In the case of a specular reflection, the transfer matrix of that kth interaction is defined as [8]

$$\mathbf{T}_k^r = \begin{bmatrix} r_\perp & 0 \\ 0 & r_\parallel \end{bmatrix} \begin{bmatrix} \mathbf{e}_{pe}^\top \mathbf{k}_u & \mathbf{e}_{pe}^\top \mathbf{k}_v \\ \mathbf{e}_{pa}^\top \mathbf{k}_u & \mathbf{e}_{pa}^\top \mathbf{k}_v \end{bmatrix}, \qquad \mathbf{e}_{pe} = \frac{\mathbf{k} \times \mathbf{n}}{||\mathbf{k} \times \mathbf{n}||_2}, \qquad \mathbf{e}_{pa} = \mathbf{e}_{pe} \times \mathbf{k},$$

where $r_\perp$ and $r_\parallel$ are the perpendicular and parallel reflection coefficients, $\mathbf{k}_u$ and $\mathbf{k}_v$ are arbitrary unit direction vectors orthonormal to the incident ray direction $\mathbf{k}$, and $\mathbf{n}$ is the surface normal. For an incident wave in vacuum, the corresponding Fresnel reflection coefficients are

$$r_\perp = \frac{\cos\theta_i - \sqrt{\epsilon_c - \sin^2\theta_i}}{\cos\theta_i + \sqrt{\epsilon_c - \sin^2\theta_i}}, \qquad r_\parallel = \frac{\epsilon_c \cos\theta_i - \sqrt{\epsilon_c - \sin^2\theta_i}}{\epsilon_c \cos\theta_i + \sqrt{\epsilon_c - \sin^2\theta_i}}.$$

Here, θ_i is the angle of incidence, and ϵ_c is the complex relative permittivity of the reflecting material, expressed as $\epsilon_c = \epsilon_r - j\,\frac{\sigma_c}{\omega\epsilon_0}$, where ϵ_r is the real relative permittivity, σ_c is the conductivity, and ϵ_0 is the vacuum permittivity.

Based on these definitions, the channel frequency response (CFR) H at frequency f is obtained with

$$H(f) = \sum_{i=1}^{N} a_i e^{-j2\pi f \tau_i}, \tag{2}$$

where N is the number of paths. In practice, Eq. 2 is sampled at uniformly spaced frequencies, forming a discrete spectrum. The equivalent CIR in the time-domain can be obtained via the inverse discrete Fourier transform. From this CIR, the power delay profile (PDP) is formed by taking the squared magnitude of each CIR sample.

3 Related Work

3.1 Differentiable Scene Representation

Differentiable scene representations have shifted from implicit neural fields such as NeRF [9] to explicit point-based methods. 3D Gaussian Splatting (3DGS) [1] replaces costly volumetric ray marching with an optimized tile-based rasterizer, enabling real-time photorealistic view synthesis. However, as standard 3DGS projects semi-transparent ellipsoids onto the image plane, it lacks the geometric structure needed to model physical transport phenomena such as multi-bounce reflections.

To address the limited physical interaction of probabilistic volumes, several works extract surface topology or enable ray intersections within the Gaussian framework. SuGaR [10] aligns Gaussians with underlying surfaces to recover continuous meshes, while 2D Gaussian Splatting [2] flattens primitives into surface-aligned disks for improved depth and multi-view consistency.

Parallel efforts introduce physically based rendering into explicit Gaussian representations. Gaussian-Shader [11] and Relightable 3D Gaussians [12] decouple materials from lighting, whereas IRGS [13] and SpecTRe-GS [14] replace rasterization with BVH-based RT to support reflections, shadows, and dynamic relighting. Hardware-accelerated RT approaches such as 3DGRT [15] and EnvGS [16] further improve efficiency and support complex reflective effects. While these methods enable visible-light rendering, their application to non-visible electromagnetic wave propagation remains largely unexplored. Building on explicit ray–Gaussian intersection, we adapt differentiable Gaussian representations to support the simulation of multi-bounce radio propagation.

3.2 Differentiable Radio Frequency Methods

Neural and non-neural scene representations have been explored for RF modeling, with NeRF [9] inspiring several channel modeling approaches. WineRT [17] predicts ray-surface interactions using a NeRF-like surrogate but relies on synthetic triangle meshes. NeRF-2 [18] combines RT with an MLP to learn spatial RF distributions, while NeWRF [19] uses direction-of-arrival cues and ray search for channel prediction with fewer measurements.

The success of 3DGS [1] has motivated explicit Gaussian-based RF field modeling. WRF-GS [20] employs spherical and Mercator projections for differentiable channel synthesis, and WRF-GS+ [21] extends this with deformable Gaussians for static and dynamic components. Complex-valued Gaussians were introduced in [22] for faster training and inference. RF-3DGS [23] adopts a two-stage optimization of radiance, geometry, and radio fields, and integrates real measurements with ray-traced simulations from synthetic meshes.

Implicit methods rely heavily on measurements and synthetic data and are expensive to train, while 3DGS-based approaches offer improved efficiency. However, except for WineRT [17], these methods cannot explicitly recover geometry-dependent propagation paths. Our method enables both novel-view synthesis and direct computation of geometric RF paths with differentiable electromagnetic modeling from an explicit radiance field.

3.3 Ray Tracing-Based Radio Propagation

Detailed handcrafted models are costly to produce, motivating the use of reconstructed 3D models for RT-based radio propagation. Triangle meshes reconstructed from point clouds are widely used [24–30], typically approximating large planar surfaces to remain compatible with ray tracers such as Sionna [31] or Wireless Insite [32].

Laser scanned point clouds offer higher geometric fidelity [33–36] but require heavy downsampling and still require multi-minute simulation times. NimbusRT v0 [37] accelerates computation via voxelization and conical rays, while NimbusRT v1 [38] models intersections as circular disks with

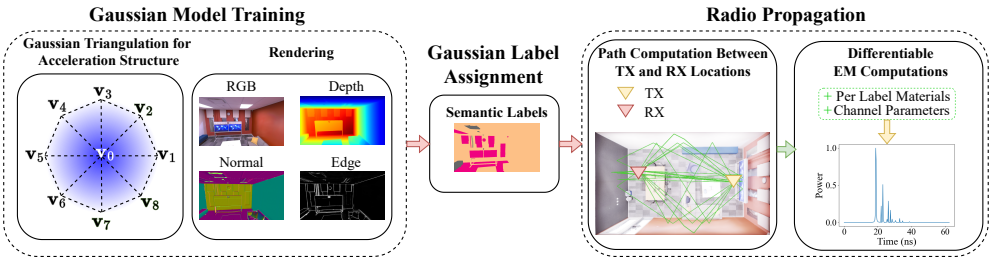


Figure 1: Our method represents 2D Gaussians using a triangle-based representation and renders RGB, normal, depth and edge images. After optimization, semantic labels are assigned to each Gaussian. The annotated model is then used to resolve propagation paths via RT, and these paths subsequently serve as input for differentiable electromagnetic computations.

distance-based attenuation and uses a visibility matrix for efficient higher-order reflections. Both rely on surface labels to prune duplicate paths and require hand-tuned primitive sizes and weights.

To enhance the accuracy of radio propagation modeling, Hoydis *et al.* [6] introduced a differentiable RT framework to calibrate the EM material properties. They use an MLP with positional encoding for improved material modeling, but it removes explicit material parameters, relies on synthetic meshes, and fails to generalize to sparse measurements. Expanding on this paradigm, [39] explored the scalability of such mesh-based differentiable pipelines for large radio environments. However, these meshes lack important geometric details relevant at higher frequencies. NimbusRT v1 [38] addressed this by supporting differentiable radio propagation with high fidelity point clouds. Despite this, its environment model remains non-differentiable, leaving the method sensitive to sensor noise and dependent on hand-tuned parameters. In contrast, our method provides a fully differentiable scene representation and uses semantic segmentation information to boost robustness under sparse channel measurement data. In addition, it supports photorealistic rendering capabilities absent in prior RT-based radio propagation simulators.

4 Method for Differentiable Gaussian Ray Tracing

Our method utilizes a set of RGB, semantic, normal, and depth images, accompanied by an optional confidence map, as well as the camera poses, and a sparse point cloud. The objective is to reconstruct a geometrically accurate 3D scene representation that supports both photorealistic novel view synthesis and physically consistent multi-bounce RT simulations. To achieve this, the point cloud is represented using 2D Gaussians, which are turned into triangle meshes for RT. We optimize them with a differentiable hardware accelerated ray tracer together with photometric and geometric regularizations. A high level illustration is provided in Fig. 1 and implementation details can be found in Appendix A.

4.1 Gaussian Representation

Our method follows 2D Gaussian Splatting (2DGS) [2], which models splats as planar Gaussians with zero extent along the z-axis. This makes them well suited for RT, since they behave as elliptical disks and allow precise ray-Gaussian intersections. Each Gaussian is defined by parameters $[\mu, \mathbf{s}, \mathbf{q}, \mathbf{h}, \sigma, \gamma]$: the mean position μ, scale $\mathbf{s} = [s_x, s_y]$, rotation quaternion $\mathbf{q}$, spherical harmonics $\mathbf{h}$ for view-dependent color, opacity σ, and edge probability γ. We additionally assign each Gaussian a material label ℓ as a post-processing step.

4.2 Ray Tracing

We first find the ray-plane intersection $\mathbf{i}$, which is followed by evaluating the Gaussian local space 2D intersection coordinates (u, v):

$$\mathcal{G} = \exp\left(-\frac{u^2+v^2}{2}\right), \qquad u = \frac{\mathbf{r}_x^\top(\mathbf{i}-\mu)}{s_x}, \quad v = \frac{\mathbf{r}_y^\top(\mathbf{i}-\mu)}{s_y},$$

where $\mathbf{r}_x$ and $\mathbf{r}_y$ are the first and second column of the rotation matrix derived from $\mathbf{q}$. We perform ray-Gaussian intersections using a hardware-accelerated ray tracer that alpha-blends Gaussians in

the exact depth order. To enable efficient intersection testing, we approximate each Gaussian with a lightweight polygonal mesh and sort intersected Gaussians in small batches of Gaussians until ray saturation, as in [15]. We cap each ray to 128 intersections and cache them to avoid a second RT pass during backpropagation. Full details of our Gaussian construction and intersection handling are provided in Appendix A.1.

Alpha Blending. Alpha blending-based rendering is performed as in standard 3DGS [1]. The ith Gaussian is evaluated for its alpha contribution ω_i in front-to-back order as follows:

$$\omega_i = T_i \sigma_i \mathcal{G}_i, \qquad T_i = \prod_{j=1}^{i-1} (1 - \alpha_j).$$

where T_i is the transmittance. Finally, the alpha blended color $\mathbf{c}$ of a ray intersecting with N Gaussians is then computed as $\bar{\mathbf{c}} = \sum_i \omega_i \mathbf{c}_i$, where $\mathbf{c}_i$ is the view-dependent color of the ith Gaussian obtained from $\mathbf{h}$. Similarly, the normal maps $\bar{\mathbf{n}}$ are computed with

$$\bar{\mathbf{n}} = \frac{\sum_i \omega_i \mathbf{n}_i}{\max\left(\epsilon, \sum_i \omega_i\right)} \tag{3}$$

where $\mathbf{n}$ denotes the Gaussian surface normal, which is the third column of the rotation matrix derived from $\mathbf{q}$, and ϵ is a small constant to avoid division by zero. For each ray, the intersection distances t_i and edge probabilities γ_i of each Gaussian are aggregated using the same weighted-sum averaging scheme as in Eq. 3, giving the distance map $\bar{t}$ and edge map $\bar{\gamma}$.

Ray Label Determination. As our application is RT, we prioritize a labeling strategy that is both explicit and computationally lightweight. We retrieve the strongest contributor among the N intersected Gaussians by selecting the one with the highest per-ray weight ω_i. This corresponds to choosing the Gaussian that contributes most to the ray under the standard 3DGS alpha blending. For semantic label generation, we use Dinov3 [40] head trained on ADE20K [41] dataset. For details on how the labels are assigned, please see Appendix A.2.

4.3 Training

The training starts from a sparse point cloud, obtained from RGB images and known camera poses. We initialize these sparse points as Gaussians, which are optimized using gradient descent. During training, our method produces alpha blended colors, depths, and normals, which are utilized in the loss functions for visual and geometric consistency. We build on prior densification and pruning strategies with minor modifications. Full implementation details are provided in Appendix A.3.

Normal Regularization. We implement normal consistency to ensure that the Gaussians are aligned with the actual surface. By assuming local planarity, pseudo normals $\mathbf{n}^d$ can be computed from the rendered depth map gradients using finite differences [2]. These depth map normals guide the Gaussian normal directions to match the underlying surface, resulting in smooth surfaces, as demonstrated in [2, 42]. To further improve the results especially near edges, gradients computed from the rgb image may be used to weight the regularization [3, 4]. Thus, following these works, we employ a normal consistency loss

$$\mathcal{L}_n = \frac{1}{N_p} \sum_i \mathcal{I}_i (1 - |\bar{\mathbf{n}}_i^\top \mathbf{n}_i^d|), \tag{4}$$

where N_p is the number of pixels and $\mathcal{I}$ is the edge weight computed from the normalized RGB image gradient as in [3]:

$$\mathcal{I}_i = (1 - \nabla c_{rgb})^2 \tag{5}$$

However, such regularization alone results in artifacts in highly reflective areas, as noted in [16, 4]. Similar to the aforementioned works, we utilize monocular normal estimates $\mathbf{n}^m$ computed using MoGe-2 [43] to supervise the rendered normals maps with

$$\mathcal{L}_m = \frac{1}{N_p} \sum_i \mathcal{I}_i (1 - |\bar{\mathbf{n}}_i^\top \mathbf{n}_i^m|). \tag{6}$$

Depth Regularization. Standard 3DGS [1] learns Gaussian parameters using only photometric supervision, which is insufficient for accurate geometry, especially in textureless or reflective regions. We add depth regularization on ray traced depth maps, following Turkulainen *et al.* [4], who show that logarithmic loss yields smoother reconstructions. When depth confidence maps are available, we further restrict the loss to pixels with maximum confidence. This gives the following loss:

$$\mathcal{L}_d = \frac{1}{N_p} \sum_i \mathcal{I}_i \mathcal{B}_i \log \left(1 + ||\bar{d}_i - d_i||_1\right), \tag{7}$$

where $\mathcal{B}_i$ is the binary mask derived from the confidence map, d_i is the ground truth depth for the ith pixel, and $\bar{d}_i$ is the estimated depth obtained from the Euclidean distance map $\bar{t}_i$.

Edge Regularization. To obtain reliable geometric edges for RT, we derive a binary edge mask from the monocular normal estimate. We compute the gradient magnitude of the predicted normals and mark a pixel as an edge whenever it exceeds the heuristic threshold t_{edge}. This gives us the binary mask $\mathcal{B}_\gamma$, which we use in our edge regularization loss:

$$\mathcal{L}_e = \frac{1}{N_p} \sum_i (\mathcal{B}_\gamma - \bar{\gamma}_i)^2. \tag{8}$$

Total Loss. We adopt the RGB reconstruction loss proposed in 3DGS [1], which combines RGB L1 loss with a D-SSIM term to better capture perceptual differences and structural consistency in the rendered images:

$$\mathcal{L}_{rgb} = (1 - \lambda_{rgb})\mathcal{L}_1 + \lambda_{rgb}\mathcal{L}_{D-SSIM}. \tag{9}$$

The combined total loss is

$$\mathcal{L}_{gs} = \mathcal{L}_{rgb} + \lambda_n \mathcal{L}_n + \lambda_m \mathcal{L}_m + \lambda_d \mathcal{L}_d + \lambda_e \mathcal{L}_e. \tag{10}$$

5 Method for Propagation Path Computation with Gaussians

The input to our method consists of the labeled Gaussian model pipeline presented in Section 4, along with the Tx and Rx positions. The objective is to identify LoS and multi-bounce propagation paths that fulfill the Fermat's principle of least time.

5.1 Path Computation

Our method is inspired by the RT-based based radio propagation simulation tool NimbusRT [38], which operates on top of point clouds. The path computation begins by identifying coarse propagation paths. These coarse paths are then refined to satisfy Fermat's principle of least time. Finally, the refined paths are post-processed to remove duplicates and ensure a clean set of propagation paths.

Coarse Path Computation. For efficient coarse path computation, we compute a visibility matrix between each RX and voxel, similar to [38]. This allows us to simply query whether an RX is visible from the intersected volume instead of an expensive RT operation. We utilize the gaussian mean points to form a voxel grid with a given resolution v_r. We refer to these voxels containing at least one Gaussian mean as discretized elements (DEs).

Once the visibility matrix is computed, we then process each TX separately. For each TX, we cast a ray to each DE, which keeps the number of rays small compared to casting potentially millions of rays. Each ray that intersects with a surface is then reflected specularly based on the intersection normal $\bar{\mathbf{n}}$. After each interaction, we then query the visibilty matrix to see whether the RX is visible from that location. In case it is, we save the path as a candidate for path refinement.

Path Refinement. The coarse paths are refined by minimizing their total length following [37, 38], assuming locally planar interactions and optimizing each interaction point in its tangent plane. The full parameterization and update rules are provided in Appendix B.

Path Pruning. When multiple refined paths converge to nearly identical interaction points, they represent the same physical specular reflection. Thus, keeping all of them would overcount its contribution. While mesh-based methods can prune duplicates via triangle indices [8], point cloud methods face noisy geometry, making existing distance- or hashing-based strategies [34, 35, 37, 38] either slow or error prone. We instead exploit the sparsity of the environment-driven RT and identify unique paths directly from geometry. For N candidates, all $N(N-1)/2$ pairs are tested: two paths are considered duplicates if each interaction point lies within distance g_d of the other point's plane defined by its normal, and if their normals differ by less than g_n. Additionally, we prune paths with interactions near edges by exploiting the edge information γ embedded in each Gaussian. Implementation details can be found in Appendix D.4.

6 Experiments

Our method is implemented using custom CUDA-based RT kernels and Slang-Torch [44]. In this section, we focus on evaluating the radio propagation simulations. The experiments were ran on a PC equipped with an NVIDIA GeForce RTX 5090 GPU. All of the parameters are provided in Appendix C.1. In Appendix D, we provide additional experiments, and show that our method achieves similar quality in novel view synthesis compared to existing Gaussian-based methods.

6.1 Validation with NLoS D-Band Channel Measurements

Measurements and Environment. We compare against the channel measurements conducted in [45]. The measurements consist of a NLoS case, where the frequency was swept in 4001 samples from 110 GHz to 170 GHz in many directional measurements. The geometry was captured with a RGB-D camera. We follow the setup provided in [37], where the measurements are composed into an aggregated CIR by choosing the strongest sample from all of the directional measurements, and all materials are assumed to be plasterboard. More details and an ablation study are provided in Appendix C.2.

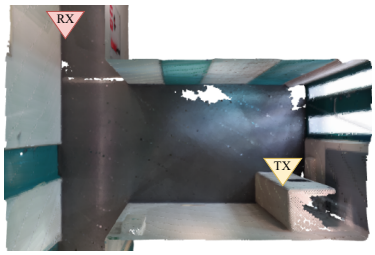


Figure 2: Corridor scene with NLOS measurements.

Baselines and Evaluation Metrics. The primary comparison class for our method is pipelines that utilize reconstructed 3D models. Thus, we compare against NimbusRT v0 [37], and its successor NimbusRT v1 [38], which are point cloud-based ray tracers for radio propagation simulation. Handcrafted mesh simulators such as Sionna [31] and Wireless Insite [32] are included as reference points under manually prepared geometry, rather than as baselines. As our evaluation metric, we utilize RMS delay spread τ_{rms}, defined as

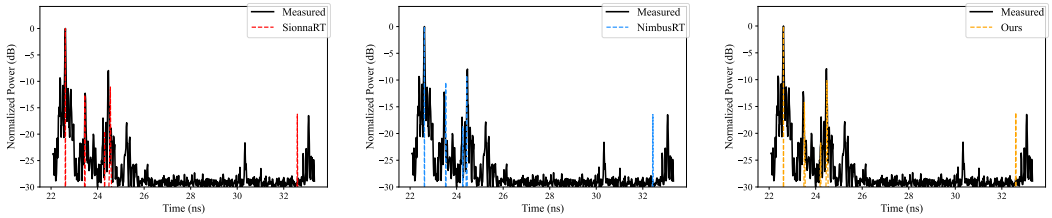
$$\tau_{\text{rms}} = \sqrt{\frac{\sum_i p_i(\tau_i - \bar{\tau})^2}{\sum_i p_i}}, \qquad \bar{\tau} = \frac{\sum_i p_i \tau_i}{\sum_i p_i},$$

where $p_i = |h_i|^2$ and τ_i are the power and delay of ith tap in the CIR, respectively, and $\bar{\tau}$ is the mean excess delay.

Results. We provide the qualitative results of simulated PDPs of Nimbus v1, Sionna and our method overlayed on top of the measured PDPs in Fig. 4. All three methods produce similar peaks as in the measured one. We further evaluate the RMS delay spread of each method in Table 1. Our method is capable of producing more similar τ_{rms} than NimbusRT in comparison to the simulations with synthetic triangle mesh model and channel measurements. This result highlights the effectiveness of a differentiable environment representation augmented with geometric regularization.

6.2 Experiments with Differentiable Radio Propagation

Our objective is to maintain a fully explicit and interpretable representation of the propagation process. We therefore optimize geometric and material parameters solely to reproduce the measured PDPs, acknowledging that many different combinations of relative permittivity and conductivity can yield similar amplitude responses and that extracting precise material properties is often difficult and time consuming. For complete details of these experiments, please refer to Appendix C.3.



(a) Sionna (b) NimbusRT v1 (c) Ours

Figure 4: Max normalized PDPs of NimbusRT v1, Sionna, and our method, compared with the measured one.

Table 1: Results of simulated τ_{rms}. The results marked with * were taken from [37].

Metrics	Measurements*	Nimbus v0*	Wireless Insite*	Nimbus v1	Sionna	Ours
τ_{rms} (ns)	1.67	1.52	1.58	1.52	1.57	1.56
Absolute Error (ns)	-	0.15	0.09	0.15	0.10	0.11

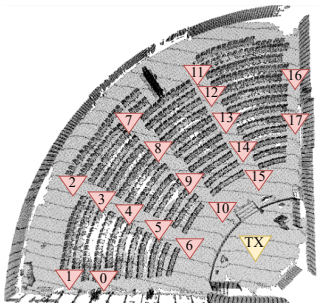


Figure 3: Auditorium scene with one TX and 18 RXs.

Measurements and Environment. The channel measurements were carried out in a large auditorium at 234 GHz center frequency. The 4 GHz bandwidth surrounding the center frequency was swept in 1001 uniformly spaced frequency samples. The setup consisted of one TX position and 18 RX positions, as illustrated in Fig. 3. The geometry was captured using a RGB-D camera.

Structural Path Alignment Loss. We begin by extracting LoS and reflection multipath components (MPCs) from noise by thresholding the measurements at -88 dB (see Fig. 5). Because adjacent taps above threshold often form clusters, we prune each cluster by retaining only its strongest MPC. We then compute the signed distance (SD) to the nearest MPC to enable reliable correspondence matching between measured and simulated paths, preventing the optimizer from locking onto weaker scatterers. A maximum SD threshold is applied to further avoid erroneous MPC associations. We formulate our loss as

$$\mathcal{L}_{spa} = \frac{1}{N}\sum_{i=1}^{N}\frac{|p_i - G_{ch}\hat{p}_i|}{p_i + G_{ch}\hat{p}_i}, \tag{11}$$

where N is the number of paths, G_{ch} is the channel gain and p_i and $\hat{p}_i$ are the ground truth and predicted powers of the ith correspondence, respectively. The power values are extracted from the measured and simulated CIRs based on the time delay of each simulated path.

Training. We train the model for 1000 iterations using the Adam optimizer with a learning rate of 1e-2. At each training iteration, we randomly pick one RX from the training set. We optimize the relative permittivity ϵ_r and conductivity σ_c, and the channel gain coefficient G_{ch}. To ensure that all material parameters remain physically valid, we optimize unconstrained variables ϵ_r', σ_c', and G_{ch}' and map them to valid values via

$$\epsilon_r = 1 + \exp(\epsilon_r'), \qquad \sigma_c = \exp(\sigma_c'), \qquad G_{ch} = \exp(G_{ch}').$$

For evaluation, we use the 18 RX locations and construct six distinct train-test splits. In each split, three RX locations are held out for evaluation while the remaining fifteen are used for training. The test RX indices follow a cyclic pattern: the first split uses RX locations $[0, 6, 12]$, the second uses $[1, 7, 13]$, and so on. In the simulation, we assume the antenna pattern to be an isotropic antenna pattern at both TX and RX side, and restrict the simulation to LoS and a maximum of two reflections.

As a baseline, we also train using the RMS delay spread–based loss introduced in [6].

$$\mathcal{L}_{ds} = \frac{|\tau_{rms} - \hat{\tau}_{rms}|}{\tau_{rms} + \hat{\tau}_{rms}} + \frac{|\mathcal{P} - \hat{\mathcal{P}}|}{\mathcal{P} + \hat{\mathcal{P}}}, \tag{12}$$

where $\mathcal{P}$ and $\hat{\mathcal{P}} = G_{ch} \sum_i p_i$ are ground truth and predicted total power, respectively.

Results. The quantitative results are provided in Table 2. The baseline loss $\mathcal{L}_{ds}$ [6] aligns closely with the evaluation metric. Thus, it can give the false impression of improved accuracy even when the underlying channel response is not well reproduced. In contrast, our loss attempts to find matching paths, providing a more faithful and physically meaningful supervision, especially if the objective is to reproduce similar reflections as observed in measurements, as shown in Fig. 5. Nevertheless, our loss $\mathcal{L}_{spa}$ on average achieves a better MAE of τ_{rms} with the test sets than the baseline. In certain cases, our method fails to find propagation paths observed in the channel measurements. An example of this can be seen in Fig. 6, leading to an absolute τ_{rms} error of 17.56 ns. Additional results for the test cases and ablation study of $\mathcal{L}_{spa}$ are provided in Appendix C.3.

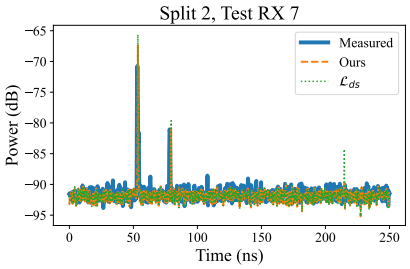


Figure 5: Unlike $\mathcal{L}_{ds}$, our loss reduces the contribution of the late arriving path to the noise level, which is not observable in the measurements.

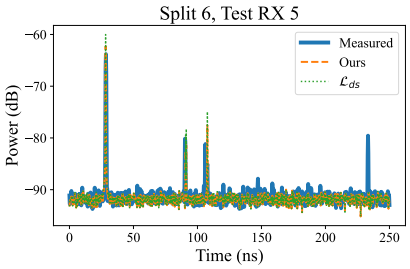


Figure 6: Our method misses a late arriving propagation path, leading to a substantial underestimation of τ_{rms}.

6.3 Discussion

Limitations. Our simulations do not consider diffraction, scattering, penetration, and temporal effects. Some paths may be missed due to the coarse RT required for duplicate path removal. Experimental validation is limited to two real-world scenes, as to the best of our knowledge, no datasets provide RGB-D images paired with channel measurements.

Broader Impact. This work integrates Gaussian-based scene reconstruction with differentiable physically grounded RF point-to-point path tracing, a step towards RF digital twins. These capabilities can benefit wireless communications, robotics, and sensing through multipath characterization, and can also be used to generate synthetic datasets for training neural networks.

Table 2: MAE of τ_{rms} per test set in nanoseconds.

Method	Set 1	Set 2	Set 3	Set 4	Set 5	Set 6	Mean
Initial	11.80	6.90	5.51	7.54	10.62	13.12	9.25
With $\mathcal{L}_{ds}$ [6]	**7.38**	6.03	3.27	6.82	**6.70**	**9.02**	6.54
With $\mathcal{L}_{spa}$ (Ours)	7.97	**4.70**	**3.20**	**5.72**	6.85	9.66	**6.35**

7 Conclusion

We introduced a unified Gaussian representation that integrates hardware-accelerated ray tracing with an explicit Gaussian scene representation, enabling accurate multi-bounce RF propagation path computation directly from visually reconstructed environments. By bridging optical reconstruction and wireless channel modeling, our approach removes the need for manual CAD geometry and supports differentiable optimization of parameters involved in radio propagation. Our results demonstrate that physically meaningful CIRs can be recovered from a photorealistic scene representation, opening opportunities for tighter integration between vision-based scene understanding and RF simulation.

Future Work. Future work includes incorporating diffraction by leveraging the per-Gaussian edge probability, adding diffuse scattering and learned antenna patterns. We also aim to stabilize path refinement and expand evaluation to larger and more diverse real-world environments.

Acknowledgments and Disclosure of Funding

We would like to thank Naveeth Lafir, Dr. Peize Zhang, and Dr. Pekka Kyösti for performing, providing, and helping with the processing of the measurement data in the auditorium scene. We also thank Dr. Joonas Kokkoniemi for the measurement data of the corridor scene and Dr. Janne Mustaniemi for capturing and reconstructing the auditorium and corridor scenes. Lastly, we thank Shakthi Gimhana, Taufiq Ahmed, Dr. Praneeth Susarla, and Dr. Dileepa Marasinghe for the setup and measurement data in the laboratory scene.

This research was supported by the Research Council of Finland (former Academy of Finland) 6G Flagship Programme (Grant Number: 346208), and the Horizon Europe CONVERGE project (Grant 101094831).

References

[1] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023.

[2] Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2d gaussian splatting for geometrically accurate radiance fields. In *ACM SIGGRAPH 2024 Conference Papers*, pages 1–11, 2024.

[3] Danpeng Chen, Hai Li, Weicai Ye, Yifan Wang, Weijian Xie, Shangjin Zhai, Nan Wang, Haomin Liu, Hujun Bao, and Guofeng Zhang. Pgsr: Planar-based gaussian splatting for efficient and high-fidelity surface reconstruction. *IEEE Transactions on Visualization and Computer Graphics*, 31(9):6100–6111, 2024.

[4] Matias Turkulainen, Xuqian Ren, Iaroslav Melekhov, Otto Seiskari, Esa Rahtu, and Juho Kannala. Dn-splatter: Depth and normal priors for gaussian splatting and meshing. In *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 2421–2431. IEEE, 2025.

[5] ITU-R P.2040. https://www.itu.int/rec/R-REC-P.2040/en. Accessed: 01.03.2026.

[6] Jakob Hoydis, Fayçal Aït Aoudia, Sebastian Cammerer, Florian Euchner, Merlin Nimier-David, Stephan Ten Brink, and Alexander Keller. Learning radio environments by differentiable ray tracing. *IEEE Transactions on Machine Learning in Communications and Networking*, 2: 1527–1539, 2024.

[7] Shuaifeng Jiang, Qi Qu, Xiaqing Pan, Abhishek K Agrawal, Richard Newcombe, and Ahmed Alkhateeb. Learnable wireless digital twins: Reconstructing electromagnetic field with neural representations. *IEEE Open Journal of the Communications Society*, 6:1568–1590, 2025.

[8] Fayçal Aït Aoudia, Jakob Hoydis, Merlin Nimier-David, Baptiste Nicolet, Sebastian Cammerer, and Alexander Keller. Sionna rt: Technical report. *arXiv preprint arXiv:2504.21719*, 2025.

[9] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021.

[10] Antoine Guédon and Vincent Lepetit. Sugar: Surface-aligned gaussian splatting for efficient 3d mesh reconstruction and high-quality mesh rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5354–5363, 2024.

[11] Yingwenqi Jiang, Jiadong Tu, Yuan Liu, Xifeng Gao, Xiaoxiao Long, Wenping Wang, and Yuexin Ma. Gaussianshader: 3d gaussian splatting with shading functions for reflective surfaces. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5322–5332, 2024.

[12] Jian Gao, Chun Gu, Youtian Lin, Zhihao Li, Hao Zhu, Xun Cao, Li Zhang, and Yao Yao. Relightable 3d gaussians: Realistic point cloud relighting with brdf decomposition and ray tracing. In *European Conference on Computer Vision*, pages 73–89. Springer, 2024.

[13] Chun Gu, Xiaofei Wei, Zixuan Zeng, Yuxuan Yao, and Li Zhang. Irgs: Inter-reflective gaussian splatting with 2d gaussian ray tracing. *arXiv preprint arXiv:2412.15867*, 2024.

[14] Jiajun Tang, Fan Fei, Zhihao Li, Xiao Tang, Shiyong Liu, Youyu Chen, Binxiao Huang, Zhenyu Chen, Xiaofei Wu, and Boxin Shi. Spectre-gs: Modeling highly specular surfaces with reflected nearby objects by tracing rays in 3d gaussian splatting. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 16133–16142, 2025.

[15] Nicolas Moenne-Loccoz, Ashkan Mirzaei, Or Perel, Riccardo de Lutio, Janick Martinez Esturo, Gavriel State, Sanja Fidler, Nicholas Sharp, and Zan Gojcic. 3d gaussian ray tracing: Fast tracing of particle scenes. *ACM Transactions on Graphics (TOG)*, 43(6):1–19, 2024.

[16] Tao Xie, Xi Chen, Zhen Xu, Yiman Xie, Yudong Jin, Yujun Shen, Sida Peng, Hujun Bao, and Xiaowei Zhou. Envgs: Modeling view-dependent appearance with environment gaussian. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 5742–5751, 2025.

[17] Tribhuvanesh Orekondy, Pratik Kumar, Shreya Kadambi, Hao Ye, Joseph Soriaga, and Arash Behboodi. Winert: Towards neural ray tracing for wireless channel modelling and differentiable simulations. In *The Eleventh International Conference on Learning Representations*, 2023.

[18] Xiaopeng Zhao, Zhenlin An, Qingrui Pan, and Lei Yang. Nerf2: Neural radio-frequency radiance fields. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking*, pages 1–15, 2023.

[19] Haofan Lu, Christopher Vattheuer, Baharan Mirzasoleiman, and Omid Abari. Newrf: a deep learning framework for wireless radiation field reconstruction and channel prediction. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org, 2024.

[20] Chaozheng Wen, Jingwen Tong, Yingdong Hu, Zehong Lin, and Jun Zhang. Wrf-gs: Wireless radiation field reconstruction with 3d gaussian splatting. In *IEEE INFOCOM 2025-IEEE Conference on Computer Communications*, pages 1–10. IEEE, 2025.

[21] Chaozheng Wen, Jingwen Tong, Yingdong Hu, Zehong Lin, and Jun Zhang. Neural representation for wireless radiation field reconstruction: A 3d gaussian splatting approach. *IEEE Transactions on Wireless Communications*, 2025.

[22] Kang Yang, Gaofeng Dong, Sijie Ji, Wan Du, and Mani Srivastava. Gsrf: Complex-valued 3d gaussian splatting for efficient radio-frequency data synthesis. *arXiv preprint arXiv:2502.01826*, 2025.

[23] Lihao Zhang, Haijian Sun, Samuel Berweger, Camillo Gentile, and Rose Qingyang Hu. Rf-3dgs: Wireless channel modeling with radio radiance field and 3d gaussian splatting. *IEEE Transactions on Wireless Communications*, 25:10419–10433, 2026.

[24] Wataru Okamura, Rikisenia Lukita, Gilbert Ching, Yuki Matsuyama, Yukiko Kishiki, Zhihang Chen, Kentaro Saito, and Jun-ichi Takada. Simplification method of 3d point cloud data for ray trace simulation in indoor environment. *IEICE Communications Express*, 9(6):182–187, 2020.

[25] Mingjie Pang, Han Wang, Kaiwei Lin, and Hai Lin. A gpu-based radio wave propagation prediction with progressive processing on point cloud. *IEEE Antennas and Wireless Propagation Letters*, 20(6):1078–1082, 2021.

[26] Han Niu, Diego Dupleich, Yanneck Völker-Schöneberg, Alexander Ebert, Robert Müller, Joseph Eichinger, Alexander Artemenko, Giovanni Del Galdo, and Reiner S Thomä. From 3d point cloud data to ray-tracing multi-band simulations in industrial scenario. In *2022 IEEE 95th Vehicular Technology Conference:(VTC2022-Spring)*, pages 1–5. IEEE, 2022.

[27] Norisato Suga, Yoshihiro Maeda, and Koya Sato. Indoor radio map construction via ray tracing with rgb-d sensor-based 3d reconstruction: Concept and experiments in wlan systems. *IEEE Access*, 11:24863–24874, 2023.

[28] Ahmad Kamari, Nan Wu, Hemant Kumar Narsani, Bo Han, and Parth Pathak. Environment-driven mmwave beamforming for multi-user immersive applications. In *Proceedings of the 1st ACM Workshop on Mobile Immersive Computing, Networking, and Systems*, pages 268–275, 2023.

[29] Guozhen Xia, Chen Zhou, Fubin Zhang, Zeyu Cui, Chengkai Liu, Hao Ji, Xinmiao Zhang, Zhengyu Zhao, and Yin Xiao. Path loss prediction in urban environments with sionna-rt based on accurate propagation scene models at 2.8 ghz. *IEEE Transactions on Antennas and Propagation*, 72(10):7986–7997, 2024.

[30] Norisato Suga, Naoya Yoshida, Ryotaro Gozono, Yoshihiro Maeda, and Koya Sato. Rgb-d sensor-aided radio map estimation using materials classification. *IEEE Wireless Communications Letters*, 2025.

[31] Jakob Hoydis, Fayçal Aït Aoudia, Sebastian Cammerer, Merlin Nimier-David, Nikolaus Binder, Guillermo Marcus, and Alexander Keller. Sionna rt: Differentiable ray tracing for radio propagation modeling. In *2023 IEEE Globecom Workshops (GC Wkshps)*, pages 317–321. IEEE, 2023.

[32] Wireless Insite. https://www.remcom.com/wireless-insite-em-propagation-software. Accessed: 01.03.2026.

[33] Usman Tahir Virk, Jean-Frederic Wagen, and Katsuyuki Haneda. Simulating specular reflections for point cloud geometrical database of the environment. In *2015 Loughborough Antennas & Propagation Conference (LAPC)*, pages 1–5. IEEE, 2015.

[34] Jan Järveläinen, Katsuyuki Haneda, and Aki Karttunen. Indoor propagation channel simulations at 60 ghz using point cloud data. *IEEE Transactions on Antennas and Propagation*, 64(10): 4457–4467, 2016.

[35] Pasi Koivumäki and Katsuyuki Haneda. Point cloud ray-launching simulations of indoor multipath channels at 60 ghz. In *2022 IEEE 33rd Annual International Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC)*, pages 01–07. IEEE, 2022.

[36] Pasi Koivumäki, Aki Karttunen, and Katsuyuki Haneda. Ray-optics simulations of outdoor-to-indoor multipath channels at 4 and 14 ghz. *IEEE Transactions on Antennas and Propagation*, 71(7):6046–6059, 2023.

[37] Niklas Vaara, Pekka Sangi, Miguel Bordallo López, and Janne Heikkilä. Ray launching-based computation of exact paths with noisy dense point clouds. *IEEE Transactions on Antennas and Propagation*, 2025.

[38] Niklas Vaara, Pekka Sangi, Miguel Bordallo López, and Janne Heikkilä. Differentiable high-performance ray tracing-based simulation of radio propagation with point clouds. *IEEE Antennas and Wireless Propagation Letters*, pages 1–5, 2026. doi: 10.1109/LAWP.2026.3670638.

[39] Stefanos Bakirtzis, Paul Almasan, José Suárez-Varela, Gabriel O Ferreira, Michail Kalntis, André Felipe Zanella, Ian Wassell, and Andra Lutu. Radio propagation modelling: To differentiate or to deep learn, that is the question. *arXiv preprint arXiv:2509.19337*, 2025.

[40] Oriane Siméoni, Huy V Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. Dinov3. *arXiv preprint arXiv:2508.10104*, 2025.

[41] Bolei Zhou, Hang Zhao, Xavier Puig, Sanja Fidler, Adela Barriuso, and Antonio Torralba. Scene parsing through ade20k dataset. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 633–641, 2017.

[42] Zehao Yu, Torsten Sattler, and Andreas Geiger. Gaussian opacity fields: Efficient and compact surface reconstruction in unbounded scenes. *arXiv preprint arXiv:2404.10772*, 2024.

[43] Ruicheng Wang, Sicheng Xu, Yue Dong, Yu Deng, Jianfeng Xiang, Zelong Lv, Guangzhong Sun, Xin Tong, and Jiaolong Yang. Moge-2: Accurate monocular geometry with metric scale and sharp details. *arXiv preprint arXiv:2507.02546*, 2025.

[44] slang-torch. https://github.com/shader-slang/slang-torch. Accessed: 01.03.2026.

[45] Joonas Kokkoniemi, Veikko Hovinen, Klaus Nevala, and Markku Juntti. Initial results on d band channel measurements in los and nlos office corridor environment. In *2022 16th European Conference on Antennas and Propagation (EuCAP)*, pages 1–5. IEEE, 2022.

[46] Zongxin Ye, Wenyu Li, Sidun Liu, Peng Qiao, and Yong Dou. Absgs: Recovering fine details in 3d gaussian splatting. In *Proceedings of the 32nd ACM International Conference on Multimedia*, pages 1053–1061, 2024.

[47] Yang Liu, Chuanchen Luo, Zhongkai Mao, Junran Peng, and Zhaoxiang Zhang. Citygaussianv2: Efficient and geometrically accurate reconstruction for large-scale scenes. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=a3ptUbuzbW.

[48] Lukas Radl, Michael Steiner, Mathias Parger, Alexander Weinrauch, Bernhard Kerbl, and Markus Steinberger. Stopthepop: Sorted gaussian splatting for view-consistent real-time rendering. *ACM Transactions on Graphics (TOG)*, 43(4):1–17, 2024.

[49] Janne Mustaniemi, Juho Kannala, Esa Rahtu, Li Liu, and Janne Heikkilä. Bs3d: Building-scale 3d reconstruction from rgb-d images. In *Scandinavian Conference on Image Analysis*, pages 551–565. Springer, 2023.

[50] Jonathan T Barron, Ben Mildenhall, Matthew Tancik, Peter Hedman, Ricardo Martin-Brualla, and Pratul P Srinivasan. Mip-nerf: A multiscale representation for anti-aliasing neural radiance fields. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 5855–5864, 2021.

[51] Peter Hedman, Julien Philip, True Price, Jan-Michael Frahm, George Drettakis, and Gabriel Brostow. Deep blending for free-viewpoint image-based rendering. *ACM Transactions on Graphics (ToG)*, 37(6):1–15, 2018.

[52] Arno Knapitsch, Jaesik Park, Qian-Yi Zhou, and Vladlen Koltun. Tanks and temples: Benchmarking large-scale scene reconstruction. *ACM Transactions on Graphics (ToG)*, 36(4):1–13, 2017.

[53] Johannes Lutz Schönberger, Enliang Zheng, Marc Pollefeys, and Jan-Michael Frahm. Pixelwise view selection for unstructured multi-view stereo. In *European Conference on Computer Vision (ECCV)*, 2016.

[54] Johannes Lutz Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[55] S. Umeyama. Least-squares estimation of transformation parameters between two point patterns. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 13(4):376–380, 1991. doi: 10.1109/34.88573.

[56] Julian Straub, Thomas Whelan, Lingni Ma, Yufan Chen, Erik Wijmans, Simon Green, Jakob J. Engel, Raul Mur-Artal, Carl Ren, Shobhit Verma, Anton Clarkson, Mingfei Yan, Brian Budge, Yajie Yan, Xiaqing Pan, June Yon, Yuyang Zou, Kimberly Leon, Nigel Carter, Jesus Briales, Tyler Gillingham, Elias Mueggler, Luis Pesqueira, Manolis Savva, Dhruv Batra, Hauke M. Strasdat, Renzo De Nardi, Michael Goesele, Steven Lovegrove, and Richard Newcombe. The Replica dataset: A digital replica of indoor spaces. *arXiv preprint arXiv:1906.05797*, 2019.

A Details of Gaussian Ray Tracing

In this appendix, we detail the components underlying our approach, including the RT details, the procedure for semantic label assignment, and the densification and pruning strategy used during Gaussian optimization.

A.1 Gaussian Mesh Construction

To enable efficient hardware-accelerated intersection testing, we approximate each 3D Gaussian with a lightweight polygonal mesh. Each Gaussian is represented by its mean μ and eight uniformly spaced vertices on the unit circle in its local tangent frame. These vertices form a coarse boundary approximation of the Gaussian support region, as illustrated in Fig. 1.

Each vertex $\mathbf{v}_i$ is transformed into world space via, similar to [15]:

$$\mathbf{v}_i = \mu + \mathbf{R}\left(\mathbf{D}_s\sqrt{\max(0,\, 2\log(\sigma))}\right)\mathbf{b}_i,$$

where $\mathbf{b}_i$ denotes the i-th unit-circle vertex in local space, $\mathbf{R}$ is the Gaussian rotation matrix, and $\mathbf{D}_s$ is the diagonal scale matrix derived from the scale parameters. Following [15], the scaling is adjusted such that the proxy boundary corresponds to the 3DGS visibility threshold of $1/255$. This polygonal proxy allows us to use hardware-accelerated ray–triangle intersection tests while for the most part preserving the spatial extent of each Gaussian.

Intersection Processing and Sorting. Our objective is to find the intersection point t with each Gaussian. As the 2D Gaussians are planar, we determine the intersection point by a ray-plane intersection with the help of the rotation matrix $\mathbf{R} = [\mathbf{r}_x, \mathbf{r}_y, \mathbf{r}_z]$ derived from $\mathbf{Q}$ with the following equation:

$$t = \frac{\mathbf{r}_z^\top(\mu - \mathbf{r}_o)}{\mathbf{r}_z^\top \mathbf{r}_d},$$

from which the intersection point can be computed with

$$\mathbf{i} = \mathbf{r}_o + \mathbf{r}_d t,$$

where $\mathbf{r}_o$ and $\mathbf{r}_d$ are the ray origin and ray direction, respectively.

Our ray tracer processes Gaussian intersections in a manner similar to [15]. For each ray, we collect up to 16 intersected Gaussians at a time, sort them by depth, and alpha-blend them in order. This procedure is repeated iteratively until the ray saturates or no further intersections are found.

Unlike prior work, we cap the number of intersections per ray to 128. All intersection results are cached and reused during the backward pass, eliminating the need for a second RT operation and significantly reducing the training time overhead, as shown in Table 7.

A.2 Semantic Label Assignment

We use Dinov3 [40] head trained on ADE20K [41] dataset to generate 2D semantic labels, which are not multi-view consistent. For each view, we compute the per-Gaussian weights ω_i, aggregate them by label, and select the label with the highest total accumulated weight. We compare against a naive assignment in which each visible Gaussian contributes equally, independent of its weight ω_i. We provide qualitative results of the labeling strategy in a scene with many objects, which leads to varying per-view 2D labels. As shown in Fig. 7, the weight-based labeling retrieves more accurate per-Gaussian labels. We also exploit this post-processing step to prune all Gaussians that were not seen in any view.

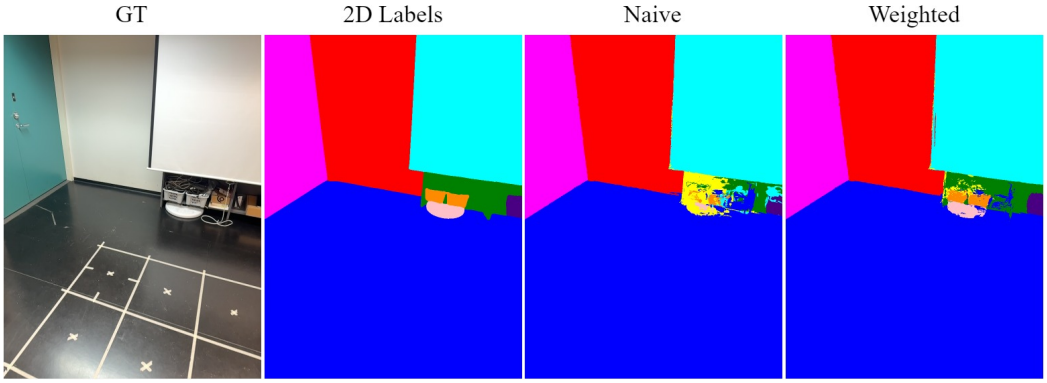


Figure 7: RGB image, 2D semantic labels, and rendered labels with naive and alpha weighted assignment.

A.3 Densification and Pruning

Densification. As the starting point of the optimization is a spare point cloud, it cannot accurately represent the whole scene. In the original 3DGS, the Gaussians are split and cloned to address the under- and over-reconstruction of the scene [1]. During the densification step, the Gaussians to be densified are picked based on the magnitude of the NDC space positional gradients of the Gaussians. However, in these methods each Gaussian may contribute to multiple pixels, leading to gradient collision, as noted by [46] and [42]. The former authors propose computing the absolute value of each gradient component, while the latter authors accumulate the norm of the individual pixel gradients. These approaches demonstrated improved reconstruction quality. For RT-based Gaussian methods, NDC space gradients do not exist, as the intersections happen in world space. This was addressed in [15] by multiplying the Gaussian mean position gradients by half of the distance to the camera. Inspired by these works, we compute take the absolute value of the position gradient, and scale it by half of the distance to the camera.

In [15], the number of primitives are limited to three million. In our implementation, we keep the number of Gaussians uncapped, which leads to our observation of densification instability, especially in outdoor scenes. We observe that this instability is caused by small Gaussians far away from the camera, where their accumulated gradients always pass the densification threshold. To counter this, we clamp the contribution of per-ray densification to $0.99t_{densify}$. In addition, we address the cases where a Gaussian may degenerate to a line, which often leads to severe over-reconstruction, producing dense Gaussian clusters. Similar behavior was reported in [47], which addressed the problem by excluding Gaussians that fulfill $\min(s_x, s_y)/\max(s_x, s_y) \leq t_s$, where t_s is the axis ratio threshold. We adopt the same filtering strategy.

Pruning. We adopt the pruning strategy of 3DGS [1], where Gaussians with $\sigma < 0.05$ are removed every 500 iterations, as our training ray tracer uses the same tile-based architecture we similarly remove the large screen space Gaussians. In addition to the opacity reset of 3DGS, we also include the opacity decay proposed by Radl *et. al.* in [48], where the opacity of each Gaussian is multiplied by 0.9995 every 50 iterations. As noted by Radl *et. al.*, this significantly reduces the number of Gaussians.

B Details of Path Refinement

To ensure that the initially detected coarse propagation paths are physically consistent and geometrically accurate, we apply a dedicated refinement stage that adjusts each interaction point to better satisfy the Fermat's principle of least time. By refining the interaction points through local optimization, we obtain paths that more faithfully represent the actual propagation behavior and can be reliably used for radio propagation modeling.

The set of coarse paths are refined by minimizing the path length as in [37, 38]. For this task, local planarity of each point is assumed, where the kth interaction point is defined as

$$\mathbf{I}_k = \mu + \mathbf{u}_k s_k + \mathbf{v}_k t_k, \tag{13}$$

where $\mathbf{u}_k$ and $\mathbf{v}_k$ are the orthonormal basis vectors of the normalized $\bar{\mathbf{n}}_k$ and s_k and t_k are the unknown parameters that are optimized. The total length of a path consisting of N interactions can is defined as

$$P = \sum_{k=0}^{N} \|\mathbf{I}_k - \mathbf{I}_{k+1}\|, \tag{14}$$

where $\mathbf{I}_0$ is the TX and $\mathbf{I}_{N+1}$ is the RX. For a path consisting of N interactions, the kth interaction point, the local path length from previous to next interaction is

$$P_k = \|\mathbf{I}_{k-1} - \mathbf{I}_k\| + \|\mathbf{I}_{k+1} - \mathbf{I}_k\|. \tag{15}$$

We then iteratively minimize P_k for $k \in [1, N]$ with respect to (s_k, t_k) using gradient descent, until the maximum number of iterations κ_i was reached, or $\|\nabla P\|^2 < t_c$, where t_c is the convergence threshold. Once a path has converged, its interaction points are validated. For the kth interaction, this is done by casting a ray from $\mathbf{I}_{k-1}$ to the optimized position. The path is accepted when two conditions hold: the angular difference between the original and updated surface normals remains

below t_a, and the perpendicular distance from the updated intersection point to the plane defined by the original $\mathbf{I}_k$ remains below t_d. If either condition is not met, the refinement is retried up to κ_r times.

C Details of Experiments

In this appendix, we provide additional details of the experiments conducted in the main paper. The experiments should be interpreted in the context of vision-derived RF simulation: the central question is whether automatically reconstructed geometry can support physically meaningful propagation paths. Manual meshes provide useful reference values, but they do not address the scalable mapping problem targeted by our method.

C.1 Parameters

Table 3 contains all parameters used throughout the paper and appendices, including their descriptions, symbols, and assigned values.

Table 3: Descriptions, symbols and values of the parameters presented in the paper.

Description	Symbol	Value
Edge magnitude threshold	t_{edge}	0.2
Voxel resolution	v_r	0.0625 m
Edge probability threshold	g_γ	0.5
Normal threshold	g_n	5°
Distance threshold	g_d	0.3 m
Normal coherency threshold	g_i	0.99
Axis ratio threshold	t_s	0.02
Refinement convergence threshold	t_c	1e-4
Refinement angle threshold	t_a	1°
Refinement distance threshold	t_d	0.01 m
Maximum refinement iterations	κ_i	50
Maximum refinement retry attempts	κ_r	10
RGB regularization coefficient	λ_{rgb}	0.2
Monocular normal regularization coefficient	λ_m	0.05
Depth normal regularization coefficient	λ_n	0.05
Edge regularization coefficient	λ_e	0.05
Depth regularization coefficient	λ_e	0.2

C.2 Corridor Scene

The channel measurements of this scene were performed in [45]. The dataset contains a NLOS scenario where the frequency is swept over 4001 samples from 110–170 GHz across many directional measurements. Following [37], we form an aggregated CIR by selecting the strongest sample across all directional measurements. The measurement setup used horn antennas with half power beamwidths (HPBWs) of 10° and 9° in azimuth and elevation, covering 90° in azimuth and 85° in elevation at the RX side. The resulting time-domain CIR has very high resolution, making it suitable for ray tracing validation. Simulations were limited to second-order reflections, and simulated paths were aligned to measurements using the strongest peak.

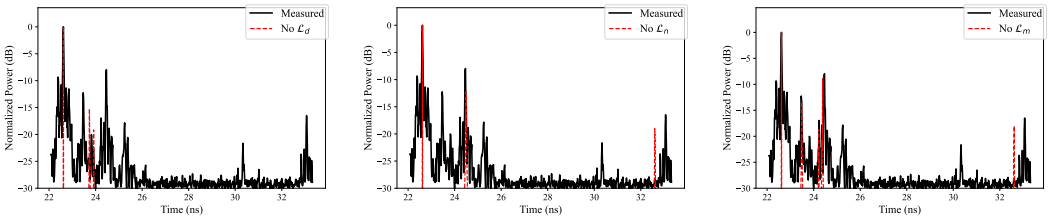
The environment was captured using Microsoft Azure Kinect camera and the poses were estimated using BS3D [49]. The dataset consists of 120 images. For a fair comparison with previous works [37], we do not perform any semantic segmentation and instead assume the same label for all Gaussians. Similarly, this label represents the electromagnetic material parameters of plasterboard at 60 GHz, as it is not defined at 140 GHz in ITU-R P.2040 [5]. We utilized MoGe2 [43] to generate the monocular normal maps. Instead of the raw sensor depth, we use rendered depths from the reconstructed triangle mesh acquired from BS3D, as the depth pixels visible in the RGB FoV was not able to capture the floors.

Results. After training, the train images had an average PSNR, SSIM, and LPIPS of 24.03, 0.907, and 0.272, respectively. These metric demonstrate mediocre training data quality, however, our method was still capable of producing accurate propagation paths, as shown in Fig. 4c. We provide qualitative rendering of some of the training images in Fig. 9

Ablations. We evaluate the effect of normal and depth regularization both in terms of the recovered propagation paths and their impact on τ_{rms}. Our ablation study isolates the contributions of depth regularization ($\mathcal{L}_d$) and the two normal-based terms ($\mathcal{L}_n$ and $\mathcal{L}_m$). The quantitative results in Table 4 show that depth regularization is particularly critical for obtaining accurate propagation paths. Although removing $\mathcal{L}_n$ results in a τ_{rms} value numerically closer to the measured one, the qualitative results in Fig. 8 reveal that several paths are missing, which ultimately increases the simulated τ_{rms}. In contrast, removing $\mathcal{L}_m$ produces paths that are visually similar to those of the full model, with only a slight increase in error. Overall, these findings indicate that $\mathcal{L}_d$ is essential for accurate geometry reconstruction, and that combining it with depth-based normal smoothing is crucial for reliable path computation.

Table 4: Qualitative results for the ablations in the corridor scene.

Metric	Measured	Without $\mathcal{L}_d$	Without $\mathcal{L}_n$	Without $\mathcal{L}_m$	Full
τ_{rms} (ns)	1.67	0.21	1.58	1.54	1.56
Absolute Error (ns)	-	1.46	0.09	0.13	0.11



(a) Without $\mathcal{L}_d$. (b) Without $\mathcal{L}_n$. (c) Without $\mathcal{L}_m$.

Figure 8: Max normalized power delay profiles of the ablation cases.

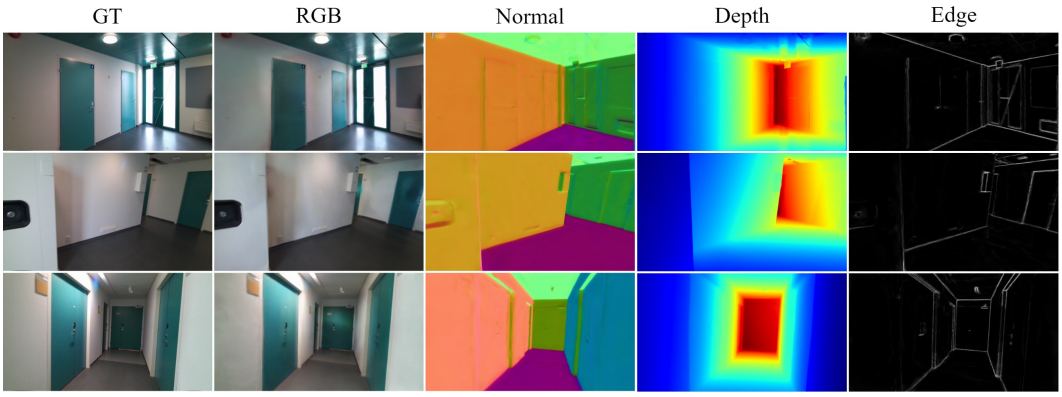


Figure 9: Ground truth RGB image, as well as the rendered RGB, normal, depth and edges by our method in the corridor scene. Zoom in for details.

C.3 Auditorium Scene

The channel measurements were conducted in a large auditorium using a VNA-based setup at a 234 GHz center frequency with 4 GHz bandwidth and 1001 uniformly spaced frequency samples. The configuration included one TX position and 18 RX positions (Fig. 3). Both TX and RX employed

(a) Color image. (b) Semantic labels from Dinov3. (c) Processed labels.

Figure 10: Label post-processing.

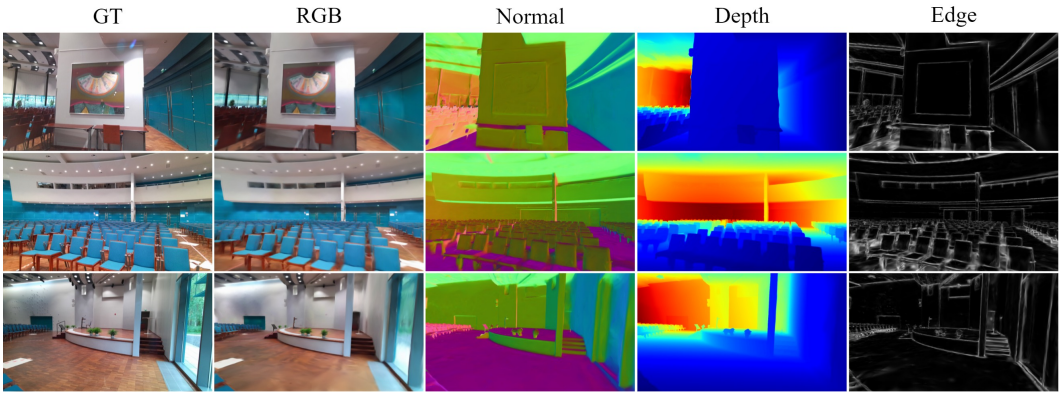


Figure 11: Ground truth RGB image, as well as the rendered RGB, normal, depth and edges by our method in the auditorium scene. Zoom in for details.

horn antennas with approximately $19.5°$ HPBW and 19.33 dBi gain. Azimuth and elevation sweeps were performed in $20°$ increments, except for the RX azimuth, which used a $10°$ step size. On the TX side, the measured azimuth spanned roughly $100°$, ensuring LoS to each RX, while the RX covered the full azimuth range. For elevation, the TX was measured only at $90°$, whereas the RX included two elevations: $70°$ and $90°$.

We utilized Microsoft Azure Kinect to collect RGB-D frames and reconstructed it using BS3D [49]. The scene consists of 1314 images which were reduced to 438 RGB, depth and normal images for training. Similar to the corridor scene, we use rendered depths from the reconstructed triangle mesh acquired from BS3D for better depth coverage, and the normals were generated with MoGe2 [43]. We also generated 2D segmentation images and assigned them to the Gaussians using the procedure described in Section 4.2. We used Dinov3 [40] head trained on ADE20K [41] dataset to generate 2D segmentation labels. To further improve the labels for the application of material parameter optimization, we processed the predicted wall labels by a simple RGB-based separation to have a separate label for the cyan and white walls (see Fig. 10).

Training Details. All of the materials were initialized with plasterboard properties computed with the equations provided in ITU-R-P.2040 [5], and the gain coefficient G_{ch} was initialized to the peak antenna gain of 19.33 dBi.

Results. The RGB-D data and the estimated camera poses exhibit substantial noise, as reflected by the average PSNR, SSIM, and LPIPS values of the training images: 20.73, 0.791, and 0.337, respectively. Qualitative results are provided in Fig. 11. Despite these challenging conditions, our method successfully recovers a significant portion of the propagation paths present in the channel measurements, as illustrated in Fig. 12.

Ablations. We evaluate the contribution of each component in our loss $\mathcal{L}_{spa}$, namely the SD, the pruning of clustered taps, and the learnable gain coefficient G_{ch} in Table 5. Without SD, the loss is forced to use the same tap index as in the measurements, even when it does not correspond to the true path. Without pruning, clustered weak taps may cause the optimizer to latch onto a non-specular

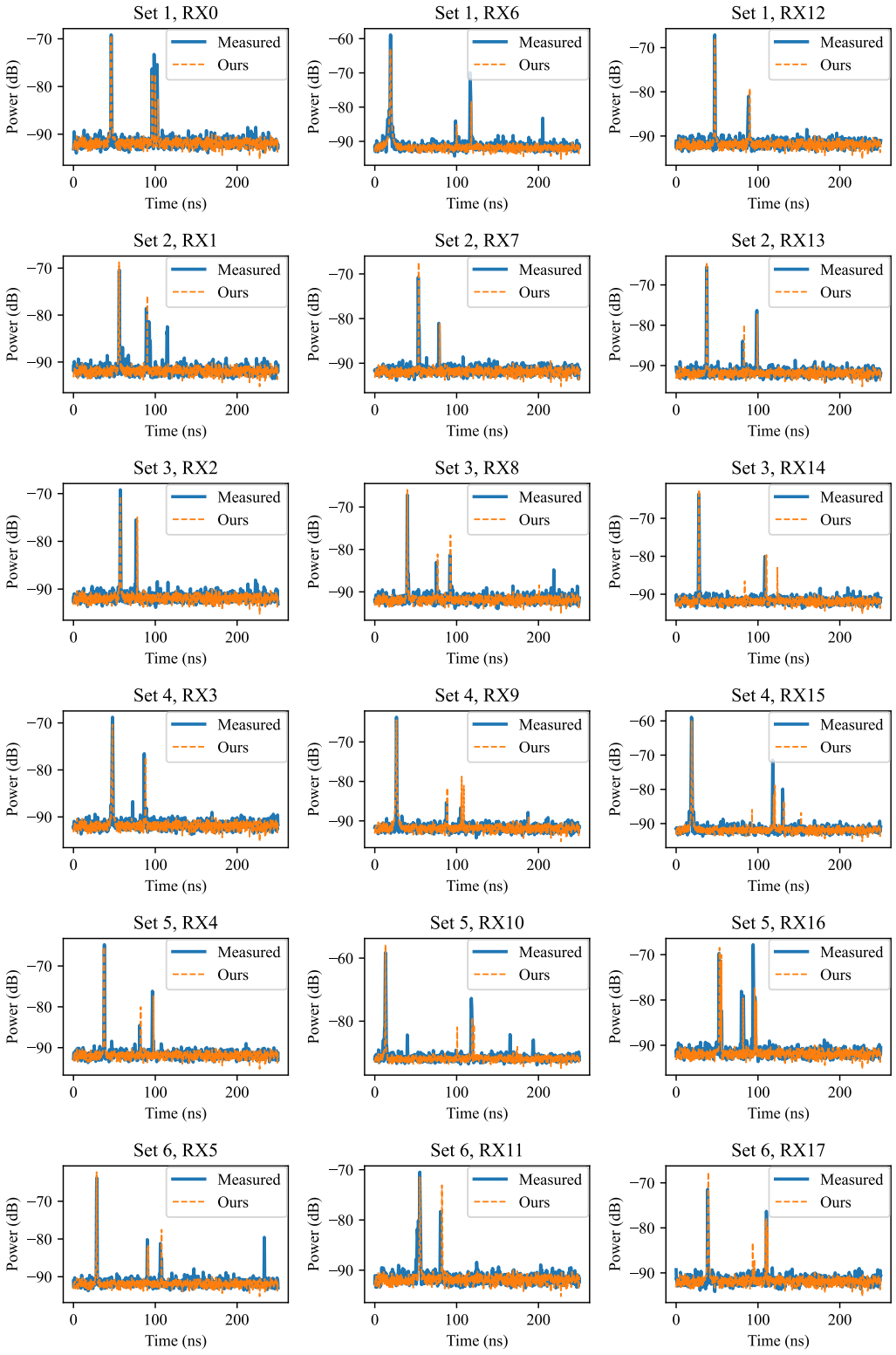


Figure 12: Qualitative results for all test RXs splits. The simulated ones are with applied noise computed from the last 50 samples of the measured RX0 PDP.

MPC instead of the dominant peak. The learnable gain coefficient compensates for deviations from the isotropic assumption, as the aggregated measurements are not perfectly isotropic.

Table 5: Ablations of components involved in $\mathcal{L}_{spa}$ and their impact of MAE of τ_{rms} in nanoseconds.

Method	Set 1	Set 2	Set 3	Set 4	Set 5	Set 6	Mean
Without SD	14.10	9.34	9.19	8.60	12.89	16.73	11.81
Without prune	11.78	7.42	5.88	7.88	10.82	12.81	9.43
Without learnable G_{ch}	9.21	5.07	3.58	6.58	8.43	11.31	7.35
Full	**7.97**	**4.70**	**3.20**	**5.72**	**6.85**	**9.66**	**6.35**

D Additional Experiments

In this appendix, we present additional experiments that further validate our approach, including novel view synthesis, synthetic radio-propagation evaluation, and an investigation demonstrating that commercial smartphones provide sufficiently rich depth data to support reliable Gaussian-based scene reconstruction for radio propagation simulation.

D.1 Novel View Synthesis

We evaluate the novel view synthesis quality of our method, as it imposes a limit of 128 Gaussians per ray and includes minor adjustments to the densification procedure. It also does not use the full Gaussian support, as we construct an approximation from 8 faces, as shown in Fig. 1.

As the datasets for validating novel view synthesis, we use Mip-NeRF360 [50] dataset, Deep Blending [51] and Tanks&Temples [52] as in previous implicit and explicit NVS papers. These datasets contain a variety of different indoor and outdoor sceneries. For each dataset, we follow the standard train/test splits used in previous works. We compare against Mip-NerF360[50] 3DGS [1], 2DGS, [2], StopThePop [48], and 3DGRT [15]. For a fair comparison, we only optimize with $\mathcal{L}_{rgb}$. As shown in Table 6, our method achieves similar quality with the baselines, while using substantially fewer Gaussians, averaging under one million per scene. We also provide qualitative results in Fig. 13, which supports the claim of similar visual quality as the baseline methods. In terms of training time, our method offers a clear advantage over 3DGRT on MipNeRF360, reducing training duration considerably, as shown in Table 7. Both methods were trained with an NVIDIA GeForce RTX 5090 GPU.

D.2 Radio Propagation Validation with Synthetic Data

To further assess the fidelity of our Gaussian-based radio propagation simulator and its ability to generalize to complex synthetic environments, we design a controlled evaluation that compares its predictions against those of Sionna [31] under identical geometric and propagation conditions. This experiment allows us to validate whether a learned Gaussian scene representation can accurately reproduce physically grounded multipath behavior. we utilize triangle mesh models provided in RF3DGS [23] dataset to compare our simulation results with Sionna. Using these meshes, we generate segmentation, normal, and depth images in addition to the RGB images included in the dataset and train our Gaussian model with these. For evaluation, we perform RT at each of the 1084

Table 6: Novel view synthesis metrics on Deep Blending, Mip-NeRF360, and Tanks&Temples datasets. We compare both implicit and explicit methods for novel view synthesis with PSNR, SSIM and LPIPS metrics. All of the results were taken from the respective papers, except the 2DGS [2] Tanks&Temples and Deep Blending results were produced by us.

Dataset	Outdoor (Mip-NeRF360)			Indoor (Mip-NeRF360)			Tanks&Temples			Deep Blending			
Metric	PSNR ↑	SSIM ↑	LPIPS ↓	PSNR ↑	SSIM ↑	LPIPS ↓	PSNR ↑	SSIM ↑	LPIPS ↓	PSNR ↑	SSIM ↑	LPIPS ↓	Avg. #Gaussians
Mip-NeRF360	24.39	0.691	0.287	31.58	0.914	0.182	22.22	0.759	0.257	29.40	0.901	0.245	-
3DGS	24.64	0.731	0.234	30.41	0.920	0.189	23.14	0.841	0.183	29.41	0.903	0.243	3.06 M
2DGS	24.34	0.717	0.246	30.40	0.916	0.195	23.13	0.832	0.213	29.57	0.904	0.257	1.75 M
StopThePop	24.46	0.722	0.254	30.03	0.917	0.194	23.18	0.839	0.184	29.84	0.905	0.241	1.54 M
3DGRT	24.34	0.721	-	30.23	0.920	-	23.20	0.830	0.222	29.23	0.900	0.315	-
Ours	24.11	0.721	0.241	29.55	0.911	0.195	22.60	0.837	0.193	30.03	0.908	0.245	0.96 M

Table 7: Average training times in Mip-NeRF360, Tanks&Temples and Deep Blending scenes.

Method	Mip-NeRF360	Tanks&Temples	Deep Blending
3DGRT	47.8 min	-	-
Ours	27.2 min	13.3 min	23.7 min

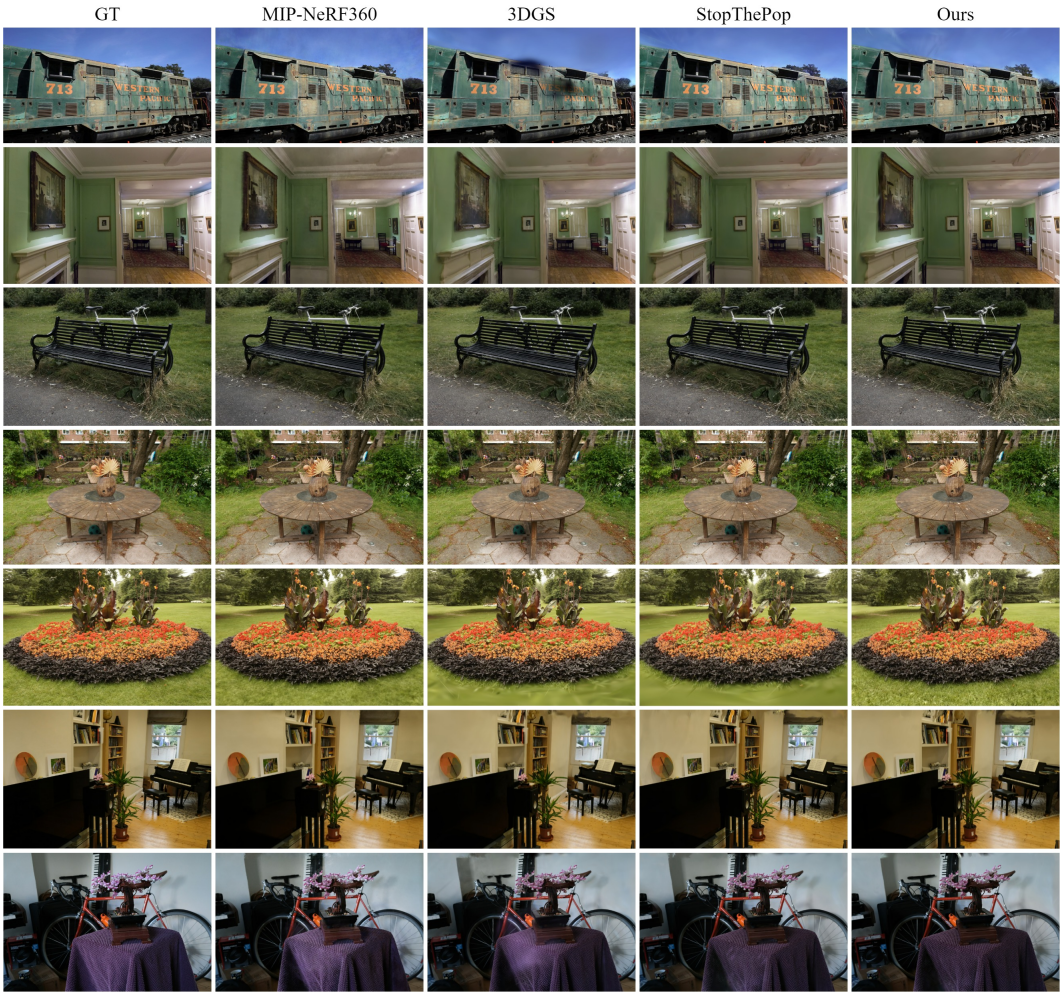
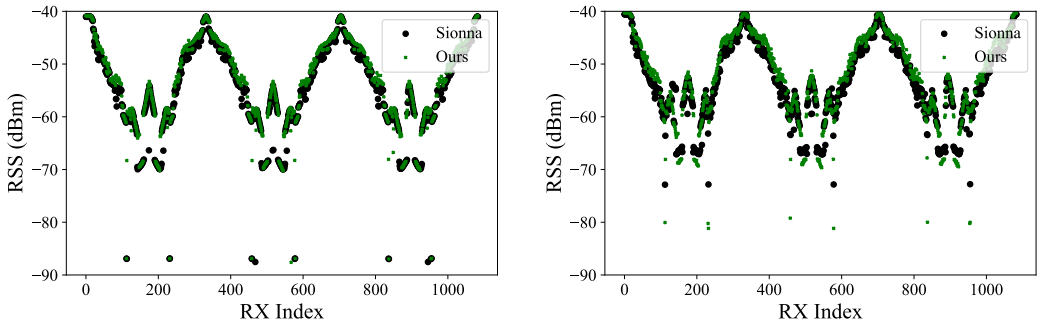


Figure 13: Novel views rendered with different methods. Mip-NeRF360 and 3DGS results were taken from [1] and StopThePop from [48].

Table 8: Comparison of mean and standard deviation of received signal strength at 1084 receiver locations.

	1 reflection & LoS		2 reflections & LoS	
	Mean (dBm)	Std (dBm)	Mean (dBm)	Std (dBm)
Ours	-53.43	8.22	-52.31	8.05
Sionna	-54.64	8.71	-53.14	7.41



(a) Received signal strength from the line of sight and first-order reflection paths.

(b) Received signal strength from the line of sight and up to second-order reflection paths.

Figure 14: Received signal strength predictions at 1084 receiver locations provided in the RF3DGS [23] dataset with our method and with Sionna.

RX coordinates provided in the dataset using both our method and Sionna [31]. From the ray traced paths we compute received signal strength (RSS) values as non-coherent sum with

$$RSS = 30 + 10 \log_{10} \left(P_{tx} \sum_{i=1}^{N} |a_i|^2 \right). \tag{16}$$

We evaluate two simulation cases, both including LoS paths: one allowing up to first-order reflections and another allowing up to second-order reflections. Qualitative and quantiative results are shown in Fig. 14, and Table 8, demonstrating agreement between our RSS predictions and those obtained with Sionna.

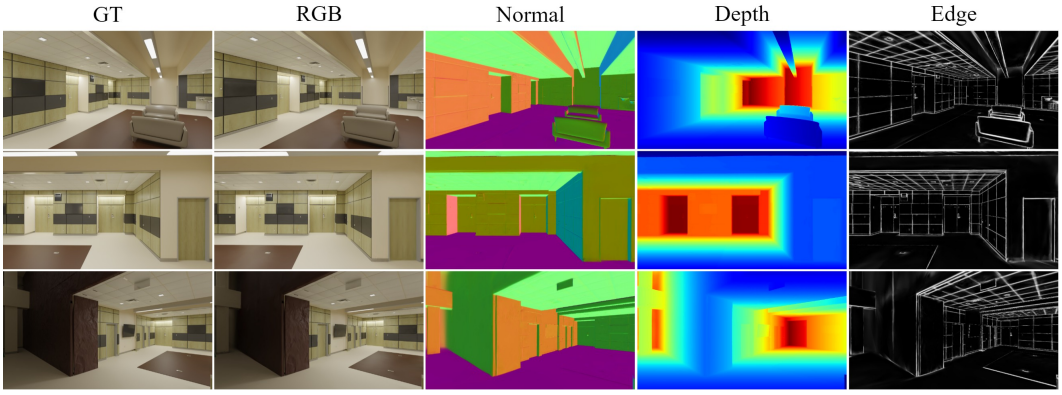


Figure 15: Ground truth RGB image, as well as the rendered RGB, normal, depth and edges by our method in the RF3DGS scene. Zoom in for details.

D.3 Real-World RSS Experiments

To enable practical and accessible scene reconstruction for wireless propagation modeling, we leverage data captured directly from a modern smartphone, demonstrating that such devices can provide sufficiently rich visual and depth information for building Gaussian-based scene representations. We

then utilize the reconstructed scene to perform RT and showcase how RSS measurements can be used to learn the channel parameters.

Scene Description. The RSS measurements were collected at each cell in Fig. 16a using two USRP N310 devices operating at a center frequency of 3.85 GHz with a 60 MHz bandwidth. Both TX and RX used dipole antennas.

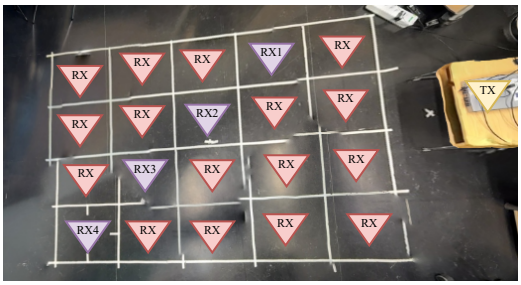
Camera data was captured with an iPhone, which uses the iPhone depth information to generate camera poses in metric scale. From our observation the camera poses were too inaccurate for Gaussian-based reconstruction, which is why we resorted to utilizing COLMAP [53, 54]. As COLMAP only uses color information, the metric scale has to be resolved separately due to scale ambiguity. Following Umeyama's least-squares similarity alignment method [55], we estimate the global rotation, translation, and scale that aligns the COLMAP camera locations to those obtained from iPhone. The semantic labels are applied to the Gaussians as described in Appendix A.2.

Training. We train the relative permittivity ϵ_r and conductivity σ_c of each material and treat the transmitted power P_{tx} as an additional optimizable parameter. The number of reflections is limited to two and the RSS is computed with Eq. (16). All material parameters are initialized to plasterboard at 3.85 GHz based on ITU-R-P.2040 [5]. The transmitted power P_{tx} is initialized to 5 dBm. As our loss function we use the mean squared error (MSE) loss

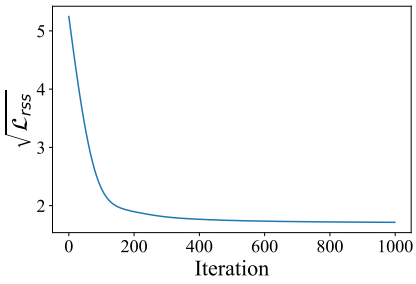
$$\mathcal{L}_{rss} = \frac{1}{N}\sum_{i=1}^{N}(RSS_i - \hat{RSS}_i)^2,$$

where RSS and $\hat{RSS}$ are theground truth and predicted RSS values. We run the training for 1000 iterations using the red RXs shown in Fig. 16a.

Results. We evaluate performance on the test set (yellow RXs in Fig. 16a). As shown in Table 9, our method achieves a mean absolute error of roughly 0.4 dBm, closely matching the measured RSS values. The loss graph in Fig. 16b further show stable and well-behaved convergence in this scenario. We also provide some qualitative results in Fig. 17.



(a) Measurement locations in the laboratory. The purple RXs (RX1-RX4) are used for validation.



(b) Per iteration loss graph for the laboratory experiments for the Red, unnumbered RXs.

Figure 16: Measurement setup in the scene and loss graph for the experiments in this environment.

Table 9: Predicted RSS values for the test set before and after optimization

RSS (dBm)	RX 1	RX 2	RX 3	RX 4
Measured	-35.7	-38.4	-42.1	-43.5
Initial	-41.4	-43.8	-46.4	-48.6
Optimized	-36.2	-38.6	-41.4	-43.7
Absolute Error	0.49	0.23	0.67	0.25

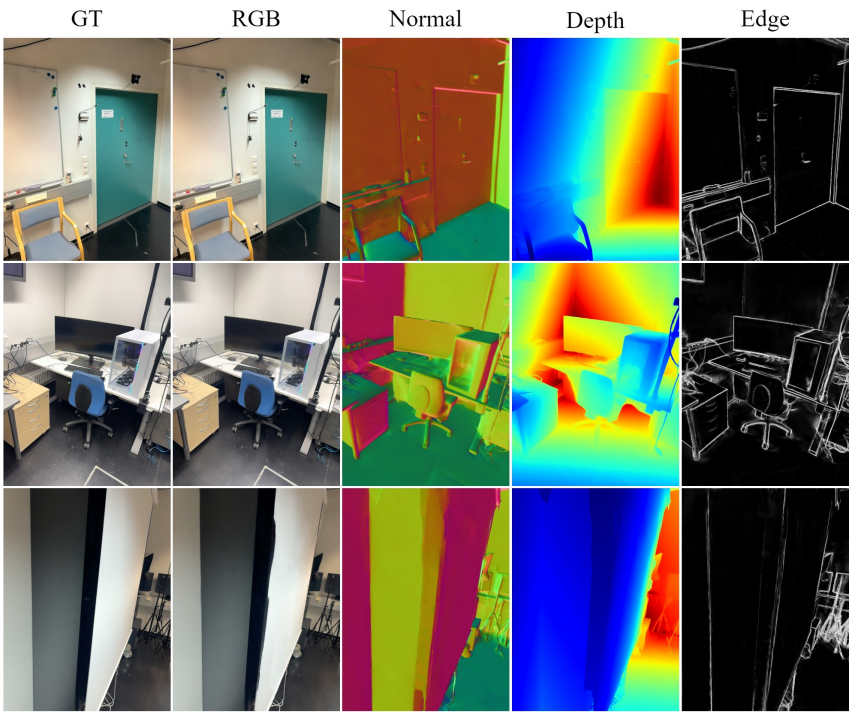


Figure 17: Ground truth RGB image, as well as the rendered RGB, normal, depth and edges by our method in the room scene. Zoom in for details.

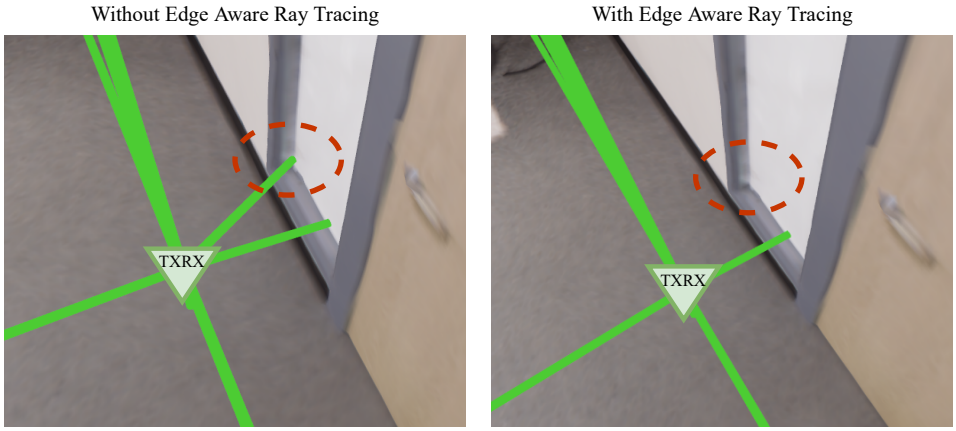


Figure 18: Ray traced paths with and without edge-aware ray tracing.

D.4 Assessment of Edge-Aware Ray Tracing

We highlight the importance of our edge-aware ray tracing by training a high-quality model on *room0* from the Replica dataset [56], which provides ground-truth depth and normal maps. In Fig. 18, we place the TX and RX at the same location and simulate only first-order reflections. Even in this ideal synthetic setting, paths can incorrectly converge to geometric edges, producing non-physical specular reflections. Our method addresses this issue by leveraging monocular normal estimates to learn per-Gaussian edge probabilities through the loss term $\mathcal{L}_e$. During point-to-point path tracing, we combine these learned edge probabilities $\bar{\gamma}$ with normal coherency $\|\bar{n}\|$ to prune invalid paths. A path is discarded if the weighted average edge probability of its intersected Gaussians exceeds the threshold $\bar{\gamma} > g_{\bar{\gamma}}$, or if the aggregated normal vector contains insufficient coherency, i.e., $\|\bar{n}\| < g_c$. This edge-aware pruning effectively removes these physically invalid specular reflections.